Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Кафедра програмування

Лабораторна робота №3

РОЗПАРАЛЕЛЕННЯ СИСТЕМ ЛІНІЙНИХ АЛГЕБРАЇЧНИХ РІВНЯНЬ

з курсу “Паралельні та розподілені обчислення”

Виконав:

студент групи ПМІ-33

Яценко Владислав Тарасович

Львів – 2025

Мета роботи
Ознайомитися з можливостями паралельного програмування у середовищі C#, реалізувати метод Якобі для розв'язання СЛАР у послідовному та паралельному варіантах, дослідити прискорення та ефективність залежно від кількості потоків, а також перевірити правильність паралельного обчислення.

Теоретичні відомості
1. Метод Якобі
Метод Якобі — це ітераційний метод для розв'язання систем лінійних алгебраїчних рівнянь (СЛАР) вигляду:

$$Ax = b$$

де A — квадратна матриця розмірності n×n, x — вектор невідомих, b — вектор вільних членів.

Основна ідея: на кожній ітерації нове значення $x_i^{(k+1)}$ обчислюється через значення з попередньої ітерації:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j=1, j\neq i}^{n} a_{ij} x_j^{(k)}\Big), \quad i = 1, 2, ..., n$$

де aii— елемент матриці на головній діагоналі.

Метод Якобі збігається, якщо матриця AAA є **діагонально домінантною**, тобто:

$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|$$

2. Послідовний алгоритм

- Ініціалізується вектор $x^{(0)}$
- На кожній ітерації обчислюється новий вектор $x^{(k+1)}$ за формулою Якобі.
- Процес зупиняється, коли максимальна різниця між $x^{(k+1)}$ та $x^{(k)}$ менша за задану точність ε.

Недолік: обчислення виконується послідовно, що при великих розмірностях матриці є дуже повільним.

3. Паралельний алгоритм
Метод Якобі добре піддається паралельній реалізації, оскільки обчислення кожного нового елемента $x_i^{(k+1)}$ **незалежне** від інших елементів цієї ж ітерації (усі вони використовують лише $x^{(k)}$.
Тому:

- Вектор невідомих можна розділити між потоками.
- Кожен потік обчислює свою частину рядків матриці.

- Після завершення всі потоки формують новий вектор $x^{(k+1)}$

Перевага — значне скорочення часу розрахунку на багатоядерних системах.

4. Прискорення та ефективність

- **Прискорення (Speedup):**

$$S_p = \frac{T_1}{T_p}$$

де T1— час виконання послідовного алгоритму, Tp— час виконання на p потоках.

Ефективність (Efficiency):

$$E_p = \frac{S_p}{p} \cdot 100\%$$

показує, наскільки добре використовуються обчислювальні ресурси.

Хід роботи

1. Реалізовано **послідовний метод Якобі**.
    - На кожній ітерації вектор невідомих x оновлюється на основі попередніх значень xOld.
    - Збіжність перевіряється за допомогою максимальної різниці між векторами x та xOld ($\varepsilon = 10^{-12}$).
    - Фіксується кількість ітерацій до збіжності.
2. Реалізовано **паралельний метод Якобі** на основі потоків (Thread).
    - Вектори xOld та xNew розділяються між потоками по рядках матриці.
    - Кожен потік обчислює нові значення лише для своєї підмножини рядків.
    - По завершенні всі потоки об'єднують результати у загальний вектор.
3. Реалізовано функцію **Verify()**, яка перевіряє, що паралельне рішення збігається з послідовним (похибка ≤ ε).
4. Для вимірювання часу використовується **Stopwatch**, а запуск повторюється декілька разів, вибирається найкращий час.
5. Матриця генерується зі штучною діагональною домінантністю (слабка, середня або сильна), що гарантує збіжність.
6. Проводяться експерименти з різною кількістю потоків — від 1 до maxThreads (кількість логічних ядер CPU).

```
Введіть розмір матриці (за замовчуванням 1000): 2000
Введіть макс. кількість потоків: 8

Виберіть рівень діагональної домінантності:
1 - Слабка (може не збігатися)
2 - Середня (збіжність за ~100-500 ітерацій)
3 - Сильна (швидка збіжність за ~10-50 ітерацій)
Ваш вибір (за замовчуванням 2): 3

Створюємо матриця 2000×2000 з рівнем домінантності 3...
Матриця створена за 77 ms

Матриця: 2000×2000, CPU: 12, тестуємо до 8 потоків
```

```
Час послідовного: 343,38 ms, ітерацій: 22

=== Результати паралельного обчислення ===
Потоки  Час (ms)      Прискорення    Ефективність   Ітерації
======================================================================
1      343,38      1.00×     100%          22
2       245,53        1,40           69,9%          28
3       168,10        2,04           68,1%          28
4       145,84        2,35           58,9%          28
5       143,73        2,39           47,8%          28
6       117,40        2,92           48,7%          28
7       119,34        2,88           41,1%          28
8       113,93        3,01           37,7%          28

=== Висновок ===
Найкраща кількість потоків: 8
Найшвидший час: 113,93 ms
Прискорення: 3,01×
Ефективність: 37,7%
```

Я запускав програму з матрицею розміром 2000×2000 і максимальним числом потоків — 8. Вибрав сильну діагональну домінантність, щоб алгоритм швидко збігався.
У послідовному режимі час виконання склав приблизно 343 мс при 22 ітераціях.
Далі я тестував паралельний варіант. На 2–4 потоках прискорення зростало, але ефективність падала. Найкращий результат вийшов при 8 потоках — час зменшився до 113,93 мс, що дало прискорення приблизно 3×, хоча ефективність становила лише близько 38%.
Паралельність реально дає виграш у швидкодії, але ідеальної лінійної масштабованості немає, бо з ростом кількості потоків з'являються накладні витрати. Найшвидше виконання було саме на 8 потоках.

Висновок:
У результаті експерименту я отримав найшвидший час при використанні 8 потоків — 113,93 мс. Це дало прискорення близько 3 разів у порівнянні з послідовним виконанням. Проте ефективність із ростом кількості потоків зменшується через накладні витрати. Оптимальним виявилося використання всіх доступних потоків.